Coding tutorial: Neural network optimization for PDEs

Xiaofeng Xu

Postdoctoral Scholar,
Department of Mathematics, Penn State University

OneMath World School
Brin Mathematical Research Center
University of Maryland
June 18, 2026

Outline

Physics-Informed Neural Networks (PINNs)

- Approximate PDE solutions with a neural network u_θ
- Enforce the governing equation at **collocation points** via automatic differentiation
- Impose **homogeneous Dirichlet** boundary conditions exactly using a hard constraint (distance function)
- Train by minimizing the PDE residual; evaluate against a known exact solution

1D Problem Setting (`pinn.py`)

Domain: $x \in (0, 1)$

PDE:

$$-u''(x) = \sin(\omega x), \qquad \omega = 10,$$

equivalently $u''(x) + b(x) = 0$ with source $b(x) = \sin(\omega x)$.

Boundary conditions: homogeneous Dirichlet

$$u(0) = u(1) = 0.$$

Exact solution:

$$u_{\text{exact}}(x) = \frac{\sin(\omega x)}{100} - \frac{x \sin(\omega)}{100}.$$

1D PINN Formulation (`pinn.py`)

Hard boundary constraint:

$$u_\theta(x) = x(1-x)\,\widehat{u}_\theta(x),$$

so $u_\theta(0) = u_\theta(1) = 0$ automatically.

Network: $\widehat{u}_\theta : \mathbb{R} \to \mathbb{R}$ is a 2-layer MLP,

$$1 \xrightarrow{\tanh} 100 \xrightarrow{\text{linear}} 1,$$

trained in `float64`.

Loss: mean squared PDE residual at 500 interior collocation points,

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\big(u_\theta''(x_i) + \sin(\omega x_i)\big)^2.$$

Optimization: Adam (6000 epochs, $\mathrm{lr} = 10^{-3}$), then L-BFGS (100 steps).
Remark: In lower dimension, you can use a uniform grid as the collocation points, while in higher dimension we have to use (quasi) Monte-Carlo points.

2D Problem Setting (pinn2d.py)

Domain: $(x, y) \in (0, 1)^2$

PDE:

$$-\Delta u(x, y) = f(x, y), \qquad \omega = 10,$$

with Laplacian $\Delta u = u_{xx} + u_{yy}$.

Boundary conditions: homogeneous Dirichlet on $\partial(0, 1)^2$,

$$u = 0 \quad \text{on } x = 0, 1 \text{ or } y = 0, 1.$$

Manufactured solution:

$$u_{\text{exact}}(x, y) = \frac{\sin(\omega x)\sin(\omega y)}{100},$$

$$f(x, y) = \frac{2\omega^2 \sin(\omega x)\sin(\omega y)}{100}.$$

2D PINN Formulation (pinn2d.py)

Hard boundary constraint:

$$u_\theta(x, y) = x(1-x)\, y(1-y)\, \widehat{u}_\theta(x, y).$$

Network: $\widehat{u}_\theta : \mathbb{R}^2 \to \mathbb{R}$ is a 2-layer MLP,

$$2 \xrightarrow{\tanh} 100 \xrightarrow{\text{linear}} 1.$$

Loss: mean squared residual on an interior tensor-product grid (50×50 collocation points, excluding the boundary),

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \big(\Delta u_\theta(x_i, y_i) + f(x_i, y_i)\big)^2.$$

Optimization & evaluation: same Adam + L-BFGS schedule as the 1D case; relative L^2 error measured on a finer 100×100 interior test grid.

Remarks

To understand the performance of the PINNs, it is important to understand how the following factors affect the convergence:

- Change the network architecture, e.g., use different activation functions, different number of layers, different number of neurons, etc.
- Change the target function to higher frequency to explore frequency principle.
- Change the number of collocation points to explore the convergence rate.

- Ma, Y., Xu, Z. Q. J., & Zhang, J. (2021). Frequency principle in deep learning beyond gradient-descent-based training. arXiv preprint arXiv:2101.00747.
- Hong, Q., Siegel, J. W., Tan, Q., & Xu, J. (2022). On the activation function dependence of the spectral bias of neural networks. arXiv preprint arXiv:2208.04924.

Summary

- Both codes solve **stationary elliptic** problems with manufactured solutions
- Boundary conditions are enforced through the hard constraint.
- PDE residuals are computed with **automatic differentiation** (second derivatives in 1D; Laplacian in 2D)

Question: Do we have other training methods to solve PDEs using neural networks that can give us better convergence?

You are encouraged to explore the following papers to gain more insights into training neural networks for solving PDEs:

- Mao, T., Xu, J., & Xu, X. (2026). Solving High-Dimensional PDEs Using Linearized Neural Networks. arXiv preprint arXiv:2601.11771.
- Xu, J., & Xu, X. (2025). Randomized greedy algorithms for neural network optimization in solving partial differential equations. Journal of Scientific Computing, 105(1), 26.
- Siegel, J. W., Hong, Q., Jin, X., Hao, W., & Xu, J. (2023). Greedy training algorithms for neural networks and applications to PDEs. Journal of Computational Physics, 484, 112084.
- Park, J., Xu, J., & Xu, X. (2022). A neuron-wise subspace correction method for the finite neuron method. arXiv preprint arXiv:2211.12031.

Thank you!